MovieLens Explorer

An Explainable Hybrid Recommendation System Using Relational and Graph Data

Chihong Yip

DSC 202 Final Project

March 11, 2026

Project Goal

MovieLens Explorer is an explainable movie recommendation and discovery tool built on the MovieLens Latest Small dataset.

This project combines:

- a **relational database element** for structured storage and analytics
- a **graph data element** for similarity, exploration, and explanation

Main tasks supported by the tool:

- personalized recommendation
- recommendation explanation
- related-movie exploration
- genre and rating analytics

Dataset

MovieLens Latest Small

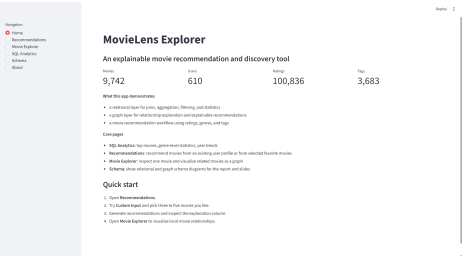
- 100,836 ratings
- 3,683 tag applications
- 9,742 movies
- 610 users

Core files

- `movies.csv`
- `ratings.csv`
- `tags.csv`
- `links.csv`

Why this dataset?

- clear structured entities



System Design

Relational Layer

- SQLite
- normalized storage
- joins and filtering
- aggregation and analytics

Core tables

- users
- movies
- genres
- movie_genres
- ratings
- tags links

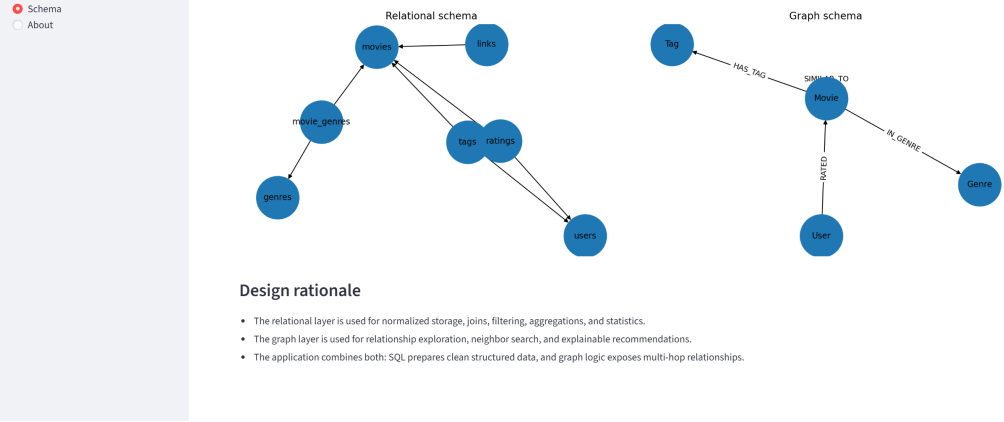
Graph Layer

- graph-based relationship modeling
- user similarity
- movie similarity
- local neighborhood exploration
- explainable recommendations

Why hybrid?

- SQL handles structure and statistics
- graph handles connections and explanation

Schema Visualization



- Left: relational schema used for normalized storage and queries
- Right: graph schema used for relationship exploration and explanation

Workflow and Scoring Logic

Pipeline

1. user selects an existing profile or favorite movies
2. relational layer retrieves metadata and statistics
3. graph layer finds similar users or related movies
4. candidates are ranked
5. explanations are returned with results

Recommendation idea

$$\text{score}(m) = \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot r_{vm} + \text{genre bonus}(m) + \text{popularity bonus}(m)}{\sum_{v \in N(u)} \text{sim}(u, v)}$$

The model is intentionally simple and interpretable rather than a black-box recommender.

Use Case 1: Recommendations

Recommendations

Recommendation mode

Existing User Custom Input

Select a user

1

Number of recommendations

5

User 1 rating history

title	rating	timestamp
Adventures of Robin Hood, The (1938)	5	964982176
Alice in Wonderland (1951)	5	964982791
All Quiet on the Western Front (1930)	5	964981497
American Beauty (1999)	5	964980868
American History X (1998)	5	964983263
American Tail, An (1986)	5	964982838
Austin Powers: International Man of Mystery (1997)	5	964981107
Back to the Future (1985)	5	964983705
Bambi (1942)	5	964980523
Basic Instinct (1992)	5	964983484

Favorite genres

genre_name	avg_rating	n_ratings
Film-Noir	5	1
Animation	4.69	29
Musical	4.682	22
Children	4.548	42
Drama	4.529	68
War	4.5	22
Adventure	4.388	85
Crime	4.356	45
Action	4.322	90
Romance	4.308	26

Deploy

Recommended movies

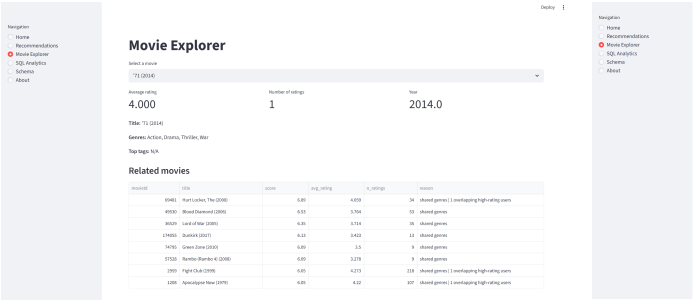
movieId	title	predicted_score	avg_rating	n_ratings	reason
91542	Sherlock Holmes: A Game of Shadows (2011)	45.776	3.763	38	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: User
6016	City of God (Cidade de Deus) (2002)	43.717	4.147	75	shared genres: Action, Adventure, Crime | liked by 1 similar users | top supporters: User 3
117646	Dragonheart 2: A New Beginning (2000)	42.853	4.5	1	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: User
6503	Charlie's Angels: Full Throttle (2003)	40.987	2.37	27	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: User
380	True Lies (1994)	39.974	3.497	178	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: User
1215	Army of Darkness (1993)	39.439	4.039	51	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: User
145	Bad Boys (1995)	38.123	3.245	51	shared genres: Action, Comedy, Crime | liked by 1 similar users | top supporters: User 162
53125	Pirates of the Caribbean: At World's End (2007)	37.694	3.438	56	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: User

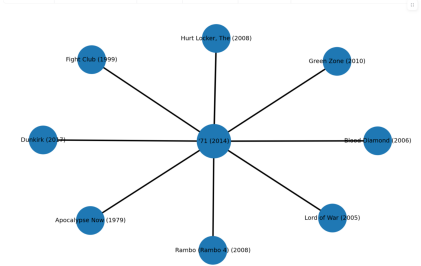
Most similar users

userId	similarity	overlap
550	1	3
1	1	232
473	0.9623	4
511	0.9258	6
9	0.9186	5
13	0.8783	6
386	0.8729	5
401	0.8659	5
535	0.8664	6
154	0.866	3

- loads user history and favorite genres
- finds similar users from overlapping ratings
- ranks unseen movies
- explains results using shared genres, similar-user support, and tags

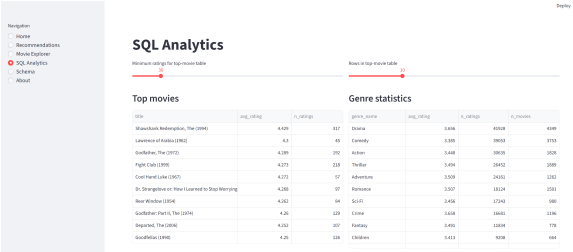
Use Case 2: Movie Explorer





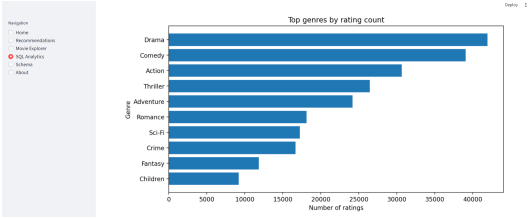
- user selects one movie
- relational layer retrieves metadata and tags
- graph layer builds a local movie neighborhood
- output includes both related-movie table and graph visualization

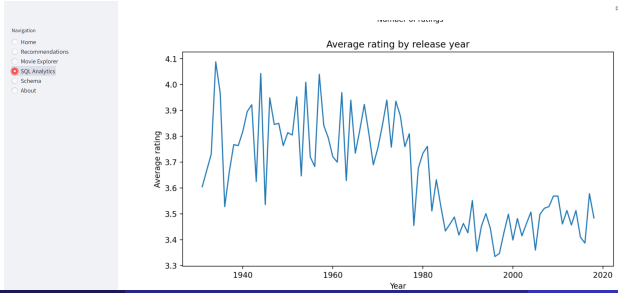
Use Case 3: SQL Analytics



title	avg_rating	n_ratings
Shawshank Redemption, The (1994)	4.429	317
Lawrence of Arabia (1962)	4.3	45
Godfather, The (1972)	4.289	292
Fight Club (1999)	4.273	218
Cool Hand Luke (1967)	4.271	57
Dr. Strangelove or: How I Learned to Stop Worrying	4.268	97
Rear Window (1954)	4.262	64
Godfather: Part II, The (1974)	4.26	129
Departed, The (2006)	4.252	107
Goodfellas (1990)	4.25	126

genre_name	avg_rating	n_ratings	n_movies
Drama	3.656	45928	4349
Comedy	3.385	39053	3753
Action	3.448	30635	1828
Thriller	3.494	26452	1689
Adventure	3.509	24351	1252
Romance	3.507	18124	1561
Sci-Fi	3.456	17243	980
Crime	3.658	16681	1196
Fantasy	3.491	11834	778
Children	3.413	9308	664





How the Work Was Completed

Step 1: Data preparation

- loaded raw CSV files with pandas
- extracted release year from titles
- normalized genre strings into separate tables

Step 2: Relational construction

- built SQLite schema
- inserted cleaned data
- added indexes for user, movie, and genre access

Step 3: Graph construction

- derived user–movie, movie–genre, and movie–tag relationships
- computed user similarity and movie similarity from behavior

Step 4: Application integration

- connected recommendation, exploration, analytics, and schema pages in Streamlit

Design Choices and Limitations

Why these choices?

- SQLite keeps the relational side lightweight and easy to query
- graph-based modeling makes similarity and explanation natural
- Streamlit makes the final tool easy to use and demonstrate

Current limitations

- limited movie metadata and no demographic user information
- recommendation score is heuristic rather than benchmark-optimized
- graph layer is implemented in Python, not an external graph database

Future work

- replace graph layer with Neo4j
- add IMDb/TMDB metadata
- add quantitative evaluation such as precision@k

Repository and Final Outputs

GitHub Repository

`https://github.com/YipGitGIT/movielens-explorer`

Repository contents

- `app.py`
- README
- dataset files
- requirements file

Final outputs

- interactive movie recommendation application
- written report
- slide deck
- GitHub repository with project code

Conclusion

MovieLens Explorer shows how a relational database element and a graph data element can be combined in a single recommendation application.

Key takeaway

- relational layer: storage, joins, and analytics
- graph layer: similarity, exploration, and explanation
- final result: a functional and interpretable recommendation tool